International Journal of Control, Automation, and Systems 23(9) (2025) 2687-2703
http://dx.doi.org/10.1007/s12555-025-0388-8
ISSN:1598-6446 eISSN:2005-4092
http://www.springer.com/12555

Modelling and Identification of a Tracked Mobile Robot: A Real-time Feasible Approach Using Particle Swarm Optimization and Information Criteria

Hamza Hobbani*, Fernando Perez-Peña, and Karsten Schmidt

Abstract: This paper presents a systematic and effective modelling and identification method for developing real-time models of a tracked mobile robot. This method uses particle swarm optimization (PSO) improved by combined Information Criteria for robust model selection. Tracked robots face unique challenges due to the complexity of their dynamics and varying terrain characteristics, thus necessitating effective model identification to reliably assist model predictive control (MPC) frameworks. Through PSO, we identify parameters for discrete state-space models across different system orders. PSO is also used to optimize the parameters for the PI controllers, which perform the chain velocity tracking. In the next step, an augmented nonlinear state-space model of the tracked robot is developed and combines the robot's dynamics with kinematic models. The selected models are validated stringently through runtime analysis to ensure they can be employed in real-time. Experimental validation shows that the identified models achieve high fidelity, with model goodness values exceeding 92% for chain velocity tracking, demonstrating the model's accuracy and suitability for real-time control. Supported by runtime calculations for the target hardware, our simulations demonstrate the validity and accuracy of the model, especially for implementation on a Raspberry Pi 4 Model B. The results show significant promise in leveraging this approach to improve trajectory tracking and control performance with real-time MPC for tracked robots, which represents an essential step towards efficient and scalable identification methods suited for real-time control problems in mobile robotics.

Keywords: Information criteria, model selection, nonlinear state-space model, particle swarm optimization, real-time control, system identification, tracked mobile robots.

1. INTRODUCTION

Due to their various applications, from industrial automation [1], agricultural tasks [2], military [3], and search and rescue missions [4], tracked (or chain-based) mobile robot control has been an active research topic. These robots need to navigate efficiently in an environment towards a target position based on pre-defined trajectories in both static and dynamic environments. Traditional control methods like PID [5-8] and sliding mode control [9-11] typically ignore the complex and time varying constraints on the tracked robots. This can cause actuator saturation, where control inputs exceed the actuator's limits. It also leads to significant tracking errors, which are amplified by track-ground interaction and terrain variations.

These limitations can be addressed using recent developments in model predictive control (MPC). MPC generates control inputs optimizing them in a receding horizon, and as such it can be applied to multi-variable, multi-constrained systems [12,13]. The predictive structure of MPC means that it can adapt control actions based on predictions of the future behaviour of the system, making it ideal for the extreme conditions. The capability of MPC to handle complexity, as well as input and state constraints, is essential for guaranteeing robust performance on variable trajectories in diverse operating conditions [14].

Nevertheless, successful MPC benefits from prior knowledge of accurate and efficient models of the robot's system dynamics and kinematics. The effectiveness of an MPC control framework models that embody the kinematic and dynamic characteristics of the robot, allowing the MPC to predict state changes and take appropriate actions in real-time. The output hysteresis and real-time constrained implementations necessitate robust models [15,16]. The models used are often accompanied by in-

Manuscript received May 8, 2025; revised July 16, 2025 and August 8, 2025; accepted August 19, 2025. Recommended by Associate Editor Seung Jae Lee under the direction of Senior Editor Won-jong Kim.

Hamza Hobbani and Fernando Perez-Peña are with the Department of Automation, Electronics and Computer Architecture and Networks, University of Cadiz, Avda. de la Universidad, no 10, 11519 Puerto Real, Cadiz, Spain (e-mails: hamza.hobbani@fra-uas.de, fernandoperez.pena@uca.es). Karsten Schmidt is with the Department of Computing and Engineering, Frankfurt University of Applied Sciences, Nibelungenplatz 1, 60318 Frankfurt am Main, Germany (e-mail: schmidtk@fb2.fra-uas.de).
* Corresponding author.

©ICROS, KIEE and Springer 2025

herent limitations, particularly with respect to those which affect the feasibility and accuracy of the optimization problem, both can hinder the practical implementation of MPC for scenarios involving dynamic, real-time applications [17-19]. This motivates the need for advanced system modelling and identification methods that can provide the accuracy and robustness required for effective MPC-based control of tracked mobile robots.

This paper offers three main contributions:

System identification using PSO with information criteria: This research presents a method of Particle Swarm Optimization and Information Criteria for improved system identification. PSO can estimate the state-space model parameters accurately, and Information Criteria can choose the optimal models regarding accuracy and robustness, thus avoiding overfitting. This approach reduces the effort of empirical testing by creating a more rigorous and usable model of the robot's dynamics, which is suitable for MPC integration.

Modelling chain dynamics and kinematics in a nonlinear state-space representation: In this work, we build a nonlinear state-space model that describes the kinematic and dynamic properties of the tracked robot, considering frictional effects, chain dynamics and damping system disturbances. This model will be used as a basis for MPC frameworks that aim for optimal trajectory tracking and robust control, and therefore, needs to be accurate and computationally efficient for target hardware implementation.

Runtime analysis and validation for MPC compatibility: An augmented state-space model is derived for the left and right chains of the tracked mobile robot and includes the dynamic of the PI-controller. The paper provides extensive computation analysis to ensure real-time feasibility. When using discrete state-space representation, we can accurately represent the dynamics of robot chains without consuming significant computational resources. In addition, this analysis is done for a Raspberry Pi 4 Model B in order to estimate the runtime of the augmented models in real time. Furthermore, this analysis was conducted on the target hardware, where the model was deployed as a ROS2 node. The experimental results confirm that the augmented models run efficiently in real time, demonstrating their suitability for MPC integration on embedded hardware.

The rest of this paper is structured as follows: Section 2 provides a review of related literature in modelling and identification methods for tracked robots. Section 3 discusses the methodology used for the system identification process. This section presents the hardware architecture, PSO implementation, application of Information Criteria for model selection, dynamic and kinematic modelling, and the derived nonlinear state-space representation. In Section 4, we outline the runtime analysis and the experimental results validating the model's suitability for MPC. Section 5 closes this research with conclusions and future research directions.

2. RELATED WORK

System identification for tracked robots: Tracked mobile robots are often chosen for missions in unstructured and rough terrains, where their traction and stability outperform conventional wheeled platforms [20-22]. However, accurately modelling such robots is challenging because their kinematics and dynamics are tightly coupled with complex track-terrain interactions [23,24]. Various approaches have been proposed to overcome these modelling difficulties, each with specific advantages and limitations [25-27]. One common strategy is to use empirical system identification to derive models from experimental data when first-principal modelling becomes intractable. For example, black-box identification methods have been applied to tracked vehicles to fit dynamic models to observed behavior under different loads and surfaces [28,29]. These data-driven models can capture highly nonlinear effects (like track slip and friction) that analytical models might omit, though they may require extensive data and careful validation. To improve identification accuracy, researchers have also explored optimization-based techniques. In particular, evolutionary algorithms have been used to estimate vehicle model parameters: In [30], a genetic algorithm is employed for off-road robot identification, while in [31] a particle swarm optimization (PSO) approach is applied to tune a skid-steer model. Such methods offer global search capabilities that help avoid local minima in parameter estimation, yielding more reliable models. Nonetheless, purely maximizing fit to data can lead to overfitting, so some works integrate model selection criteria into the identification process. For instance, authors in [32] incorporate information-theoretic criteria during identification to balance model complexity and accuracy, ensuring the chosen model generalizes well. Overall, prior studies show that combining advanced optimization with rigorous validation is key to identifying track robot models that remain faithful across the varying conditions these robot's encounter.

Dynamic modelling and kinematics: Aside from purely data-driven methods, many researchers have pursued physics-based modelling of tracked robots to capture their motion mechanics. Early works established kinematic models by extending differential-drive formulations to tracked systems, accounting for track contact geometry and skid steering effects [33,34]. These models provide a simplified description of motion (e.g., relations between track velocities and vehicle turning radius) but often assume idealized conditions (no slippage or deformation). To address this, more comprehensive dynamic models have been developed that include the track-soil interaction forces and internal track dynamics. Classical

terramechanics approaches model the track-ground contact using empirical formulas or pressure-sinkage relationships, effectively blending robot dynamics with soil mechanics [23,24]. For example, [35] presents a detailed multi-body model of a tracked vehicle with an integrated soil model to predict traction and sinkage on soft ground. Similarly, the model in [36] introduces nonlinear friction laws to represent track slippage on various terrains, improving fidelity in rough conditions. These high-fidelity models tend to predict robot behaviour more accurately over a range of terrains and manoeuvres, but they come at the cost of increased complexity and computational load [37]. Because of this, researchers have also explored simplified or reduced-order models that retain key dynamic effects while omitting less critical details. In [38], a planar dynamic model is derived for a tracked robot, capturing the essential mass distribution and drivetrain dynamics but linearizing the track-terrain contact forces. Likewise, [39] formulates a reduced state-space model for the track chain dynamics (including sprocket inertia and damping) to enable faster simulation and control design. These simplified models are easier to calibrate and run in real time, though they may lose accuracy if the robot operates in regimes outside their calibration data. A recent trend is to combine first-principles modelling with system identification - sometimes called grey-box identification - to leverage known physics while estimating uncertain parameters from data. Studies in [40-42] follow this approach, starting with a physics-based template (for example, a nominal skid-steer kinematic model) and then identifying correction factors or extra dynamics from experimental tests. This hybrid methodology often produces models that strike a practical balance between fidelity and complexity, which is especially important for designing controllers on computationally constrained platforms.

Real-time control and model deployment: For tracked robots, having an accurate model is only part of the challenge - the model must also be usable for real-time control on embedded hardware. Prior work on model-based control of tracked vehicles has demonstrated the benefits of using detailed models in guidance and control algorithms. For instance, model predictive control (MPC) and other advanced strategies have been applied to tracked robots to improve trajectory tracking in challenging terrain [43-45]. These approaches use the robot's dynamic model to predict future behaviour, enabling proactive adjustment of track speeds to follow a desired path. In [46], an MPC scheme is implemented on a tracked robot platform, showing superior tracking performance compared to conventional PID by accounting for track slip and actuator limits. Similarly, adaptive and robust controllers have been designed using identified models to handle uncertainties in payload or terrain; for example, [47] uses an adaptive term based on the model error to maintain stability on uneven ground. A major concern in these studies is computational feasibility: high-fidelity models and exhaustive prediction horizons can make real-time execution difficult on the small onboard computers typically used in mobile robots. Researchers have therefore explored several strategies to ensure real-time deployment. One strategy is code optimization and leveraging faster numerical solvers, as noted in [48], which reports achieving real-time control updates by efficiently computing the model equations and Jacobians. Another common approach is model reduction for control purposes - for example, using a lower-order model for the controller while keeping the detailed model offline for simulation [49]. By simplifying dynamics (e.g., ignoring fast but minor dynamics of the track tension system), the control algorithm can run within the tight time budgets of onboard processors. Recent experimental studies have validated the viability of deploying sophisticated controllers on low-cost hardware. In [50], a nonlinear state-space model of a tracked robot (including both chains and a PI velocity controller model) is implemented on a microcontroller, and its update rate is confirmed to meet the real-time requirements of the control loop. The authors of [51] further demonstrate that even with an advanced MPC algorithm, an embedded computer like the Raspberry Pi can handle the computations if the model and horizon are chosen judiciously. They report successful real-time trajectory tracking on a tracked robot using a Raspberry Pi-based controller, with the model update and optimization completing within each 100 ms control cycle. Integrating such controllers into practical robotic systems has also been facilitated by robotic middleware. Using frameworks like ROS2, models identified off-line can be deployed as part of the robot's control software, as described in [52]. This allows seamless integration of the model into the robot's architecture for tasks such as state estimation, feedforward control, and fault detection.

In summary, the related work shows that researchers have progressively moved towards models and identification methods that not only describe tracked robot dynamics with high accuracy but can also be executed on real hardware in real time. These developments in system identification [28-32], dynamic modelling [33-39], and real-time implementation [43-52] collectively underpin the methodology and objectives of our work. Here, we build on this literature by using PSO-assisted identification to obtain an accurate yet computationally efficient model of a tracked robot, and we demonstrate its deployment in an MPC framework on embedded hardware, bridging the gap between detailed modelling and practical control.

3. METHODOLOGY

Our methodology combines hardware experimentation with algorithmic identification and modelling. First, we describe the hardware architecture used to collect data and

validate the models. We then detail the PSO-based system identification procedure, including how candidate models are evaluated with information criteria. Next, we present the derivation of the robot's dynamic and kinematic models, which provide the structure for system identification. Finally, we show how these components are integrated into a nonlinear state-space model suitable for real-time control.

3.1. Hardware architecture

Fig. 1 illustrates the hardware architecture of the tracked mobile robot used in our research. The rover used in this study is the same as the one described in [53]. This platform provides a robust platform for developing and testing advanced control algorithms for tracked mobile robots. The system is divided into two main components: the ground station and the Rover Horizon XIX.

The ground station is the control hub for Rover Horizon XIX and consists of a PC and a network antenna. A Linux distribution is installed on the PC where Robot Operating System 2 (ROS2) Foxy [54] enables reading signals from the rover system and sending control commands in teleoperation mode. The ROS2 framework supports modular and scalable development by integrating different sensors, actuators, and control algorithms. The network antenna (Ubiquiti Rocket M2 [55]) ensures the secure connection between ground station and rover using a static IP address.

A perspective view of the Horizon XIX chassis is represented in Fig. 2 while Table 1 shows the relevant parameters of the mobile robot. The rover is equipped with

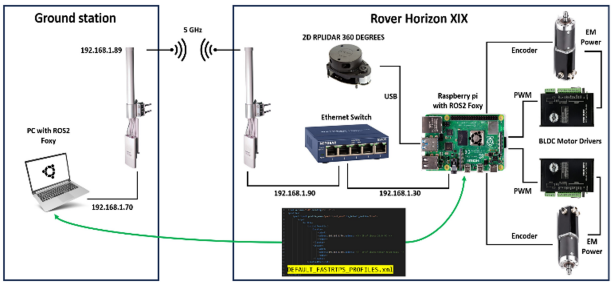


Fig. 1. Hardware architecture of the ground station and Rover Horizon XIX for tracked mobile robot control.

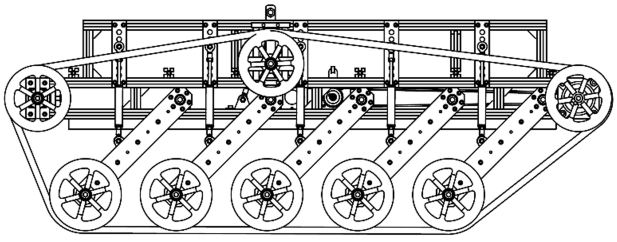


Fig. 2. Side view of the Horizon XIX chassis design.

Table 1. Relevant parameters of the Horizon XIX tracked mobile robot.

Parameter	Value
Weight	53 kg
Length	1270 mm
Width	788 mm
Height	445 mm
Chassis clearance	220 mm
Traction wheel diameter	138 mm

various hardware components in order to navigate and to perform tasks in diverse environments. The main elements of the rover include the same Rocket M2 network antenna that facilitates wireless communication with the ground station. A 2D RPLIDAR sensor is connected via USB to provide environmental mapping data. This sensor is essential for navigation and obstacle avoidance. The rover also features a Raspberry Pi 4 with 8 GB RAM [56] running a Linux distribution with ROS2 Foxy. This unit serves as the main real-time computing system and handles sensor data processing and executing control commands. The Netgear ethernet switch [57] connects all devices and ensures communication The rover is powered by two Brushless DC motors. These motors are controlled via motor drivers (ACT BLDC Motor Drivers) that manage power and speed. The encoders included in the motor units provide real-time feedback of position and speed. Pulse Width Modulation signals are used to control the BLDC. An emergency power system ensures all components have a reliable power supply and can quickly shut down the rover in case of an emergency. Additionally, the DEFAULT_FASTRTPS_PROFILES.xml configuration file defines the quality of service (QoS) settings for ROS2 communication. This is used to ensure reliable and efficient data exchange between the ground station and the rover.

The combination of these components enables the Rover Horizon XIX to operate autonomously or via remote control. The simple and robust hardware architecture, supported by ROS2 Foxy, provides a flexible and scalable platform for developing advanced control algorithms and integrating new sensors or actuators as needed.

3.2. Particle swarm optimization

Particle swarm optimization (PSO) is a robust global optimization technique developed by Kennedy and Eberhart in 1995. This technique is inspired by the social behaviour of animals such as bird flocking and fish schooling and is designed to efficiently optimize linear and nonlinear functions by simulating the movement and intelligence of a swarm of particles [58].

A swarm consists of m particles, where each particle represents a potential solution in the search space. At itera-

tion k, each particle is characterized by its current position $\boldsymbol{p}_k$, its velocity $\boldsymbol{v}_k$, and the best position it has achieved so far, known as $\boldsymbol{p}_{best,k}$. The dimension D of these vectors is determined by the number of parameters to be optimized [58,59]. The position of the x-th particle at iteration k is given by

$$\boldsymbol{p}_k^x = [p_k^{x,1}, \ldots, p_k^{x,y}, \ldots, p_k^{x,D}]^T. \tag{1}$$

Initially, the positions and velocities of the particles are randomly initialized within predefined bounds. The particles then move through the search space and tune their velocities and positions based on their own experience and the experience of their neighbours [59,60]. The velocity update rule combines cognitive and social components, as follows:

$$\begin{aligned} v_{k+1}^{x,y} &= \psi_0 v_k^{x,y} + \psi_1 \tau_k^{x,y}(p_{best,k}^{x,y} - p_k^{x,y}) \\ &\quad + \psi_2 \tilde{\tau}_k^{x,y}(p_{glob,best,k}^{x,y} - p_k^{x,y}), \\ p_{k+1}^{x,y} &= p_k^{x,y} + v_{k+1}^{x,y}. \end{aligned} \tag{2}$$

Here, $\tau_k^{x,y}$ and $\tilde{\tau}_k^{x,y}$ are random variables uniformly distributed in $[0, 1]$, and ψ_0, ψ_1, and ψ_2 are parameters that control the influence of the cognitive and social components. Typically, ψ_1 and ψ_2 are set to 2.0 to balance exploration and exploitation, while ψ_0 (inertia weight) is gradually reduced from values greater than 1.2 to values around 0.8 to ensure convergence [58-60]. In this study, PSO is implemented to identify discrete linear state-space models for the left and right chain dynamics with multiple system orders, represented as follows:

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k, \\ y_k &= Cx_k + Du_k. \end{aligned} \tag{3}$$

Once multiple candidate models with various orders are identified, selecting the best model becomes essential. A combination of information criteria, including the Akaike information criterion (AIC) and the Bayesian information criterion (BIC), is used for this purpose.

Akaike information criterion (AIC): This criterion balances model fit and complexity by penalizing models with more parameters

$$AIC(k) = 2k - 2\ln L, \tag{4}$$

where k is the number of parameters and L is the likelihood of the model [61,62].

Bayesian information criterion (BIC): Like AIC, but with a stronger penalty for models with more parameters, especially as the sample size n increases

$$BIC(k) = k\ln(n) - 2\ln(L). \tag{5}$$

BIC is preferred when the sample size is large, as it tends to select simpler models [63].

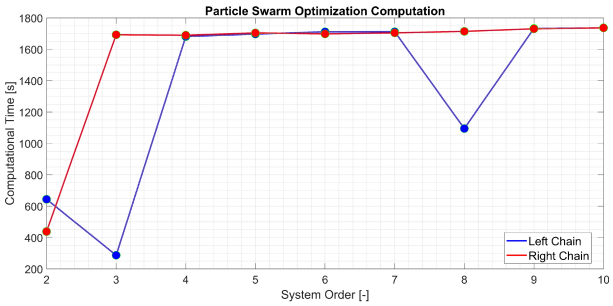


Fig. 3. Computational time of particle swarm optimization across system orders for left and right chains.

For selecting the most appropriate order for the PSO identified state-space model of the left and right chain dynamics, AIC and BIC values were calculated for various model orders to balance simplicity and accuracy. A combined criterion is used in this research and is defined as follows:

$$BC = \alpha * BIC + (1 - \alpha) * AIC, \tag{6}$$

with $\alpha = 0.5$, unifies the advantages of both criteria and allows a balanced evaluation of model order. This approach enables selection of a model that captures essential dynamics while minimizing unnecessary complexity.

Fig. 3 shows the computational time required for Particle Swarm Optimization across different system orders for the identification of the left and right chains of our tracked mobile robot. The x-axis represents the system order, ranging from 2 to 10, while the y-axis denotes the computational time in seconds. The blue line corresponds to the computational time for the left chain, and the red line corresponds to the right chain. Each point on the graph represents the time taken for PSO to converge to a solution for the optimization of each system order. The computational time measured generally increases with system order for both chains and reflects the added complexity as the model order increases. Notably, there are two significant deviations from this trend. The first occurs at system order 3, where there is a sharp drop in computational time for the left chain which indicates a potential reduction in computational complexity or faster convergence behaviour at this order. The second, more prominent anomaly is observed at system order 8, where the computational time, again for the left chain, decreases considerably. The computational time anomaly at system order 3 and 8 might be due to the convergence behaviour of PSO, as PSO performance can vary based on the dimensionality and stability of the system it attempts to optimize.

For certain system orders, the PSO algorithm may encounter simpler optimization landscapes which lead to faster convergence. This phenomenon, where specific dimensions or configurations allow faster computation, has

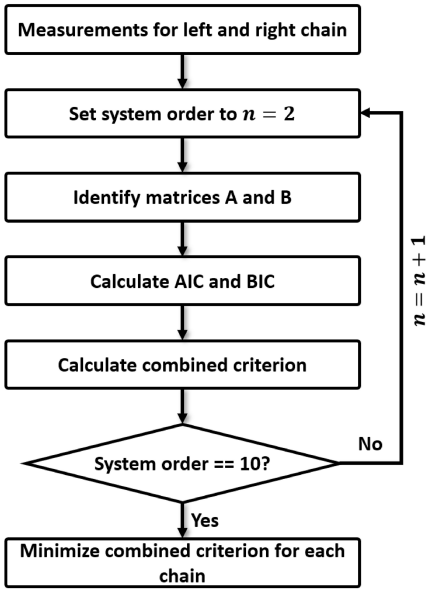


Fig. 4. Overview of the system identification procedure.

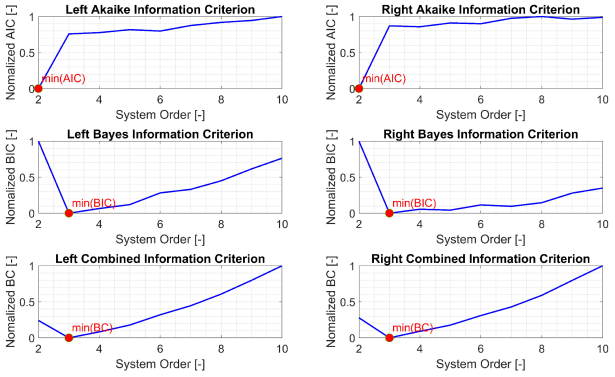


Fig. 5. Evaluation of system order based on Akaike, Bayesian, and combined information criteria for left and right chain dynamics.

been noted in optimization literature [64]. For example, when the search space aligns more closely with the solution in terms of dimensionality, optimization algorithms like PSO may converge faster due to reduced complexity in navigating the solution space. Possible explanation involves the relationship between system order and the robot's inherent dynamics. At specific orders, particularly near the optimal model order, fewer iterations might be required to minimize the objective function, thereby reducing computation time. Additionally, certain system orders could inadvertently introduce more redundant or linearly dependent variables. This simplifies the optimization process and enables PSO to resolve the identification problem more efficiently.

Fig. 4 provides a visual summary of our system identification methodology. We begin with experimental data from the left and right chains and perform PSO-based identification of the state-space matrices Aand B across model orders from 2 to 10. For each order, AIC, BIC, and a combined criterion are computed. After the loop is complete, the optimal model order for each chain is selected by minimizing the combined criterion. This structured approach ensures balanced model accuracy and complexity, while avoiding overfitting.

Fig. 5 presents the system order evaluation based on AIC, BIC, and the combined criterion (BC). Measurements were accomplished by operating the robot in teleoperation mode, with joystick inputs linearly mapped to motor commands using PWM.

In this figure, each subfigure shows the normalized values of these criteria across the different model orders. The top row illustrates the AIC values for the left and right chains, where the minimum AIC values are marked to indicate the preferred model order according to this criterion. Similarly, the middle row presents BIC values, with the minimum BIC marked as well. Finally, the bottom row displays the combined BC and incorporates aspects of both AIC and BIC to highlight the model orders that minimize this combined criterion. The results indicate that, for the left chain, AIC reach its minimum at model order 2 and BIC at model order 3. The combined BC also shows a minimum at order 3, suggesting this model order as optimal based on multiple criteria. On the other hand, for the right chain, AIC prefers again order 2 and BIC favours a model order of 3. However, the combined criterion yields a system order of 3.

In conclusion, based on the evaluation of AIC, BIC, and BC for both chains, the model with order 3 is selected as the most suitable for accurately describing the dynamic behaviour of the tracked mobile robot.

3.3. Chain dynamics

To develop the chain dynamics model, ensuring canonical observability was considered in the parameter identification. Observability allows the reconstruction of internal states using only the output measurements [65]. The state matrix A is structured in canonical form meaning it follows a standard representation that simplifies analysis while preserving observability properties. This structure ensures that the system's states can be fully reconstructed from its outputs as shown in

$$A = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_0 & a_1 & a_2 & \cdots & a_{n-1} \end{bmatrix}. \tag{7}$$

In this structure, the first $n-1$ rows are filled with zeros and ones, and the last row contains the system parameters $a_0, a_1, ..., a_{n-1}$. This form reveals whether the system can be fully observed based on the outputs. Consequently, the

number of parameters that need to be identified for each system order is given by

$$Param_{num} = 2 * System_{order}. \tag{8}$$

The factor 2 accounts for parameters in both the last row of matrix A and the complete matrix B, which defines the input-to-state relationship. This approach reduces the number of unknowns compared to a full identification of all elements in matrix A. By focusing only on the last row of matrix A and the matrix B, we improve computational efficiency while maintaining system observability.

Considering the canonical observability in (7), the identified discrete state-space models for the chain dynamics are represented for the left side in (9) and for the right side in (10). These models are structured with a sample time of $T_s = 0.1$ s to allow real-time application. For the left chain, the model is

$$\begin{aligned} x_{k+1} &= \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0.4822 & -1.364 & 1.723 \end{bmatrix} x_k + \begin{bmatrix} 0.1645 \\ 0.1577 \\ 0.0729 \end{bmatrix} u_k, \\ y_k &= \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} x_k. \end{aligned} \tag{9}$$

Similarly, for the right chain, the discrete state-space model is

$$\begin{aligned} x_{k+1} &= \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0.4956 & -1.241 & 1.716 \end{bmatrix} x_k + \begin{bmatrix} 0.1316 \\ 0.2262 \\ 0.1056 \end{bmatrix} u_k, \\ y_k &= \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} x_k. \end{aligned} \tag{10}$$

It is worth noting the subtle differences in the identified parameters between the left and right state-space models. Specifically, the matrix elements in the state transition and control matrices show slight variations. For instance, in the state transition matrix, the value at the third row and first column is 0.4822 for the left chain (9) and 0.4956 for the right chain (10), and the values at the third row and second column differ slightly as well: -1.364 for the left and -1.241 for the right. Similarly, in the control matrix, parameters differ in values, with elements such as 0.1645 for the left chain's first row and 0.1316 for the right chain's first row. These small variations indicate minor but potentially impactful differences in how each chain responds to control inputs, which should be considered in the control design and real-time application of the model.

Fig. 6 illustrates the validation of the third-order models identified for the left and right chains of the tracked mobile robot and compares the measured and predicted velocities over time. In these plots, blue lines represent measured velocities, while red lines show the model predictions. The high degree of overlap between the measured data and the model outputs suggests that the identified models accurately capture the dynamics of each chain.

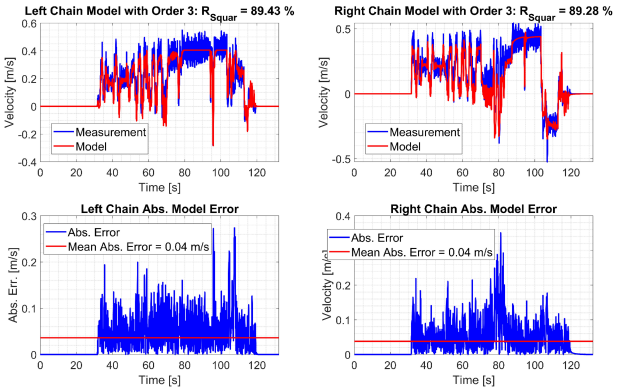


Fig. 6. Model verification and tracking error for left and right chains at system order 3.

To provide a clearer quantification of the model fidelity, we additionally plotted the tracking error (difference between measured and predicted velocities) for each chain in Fig. 6. The mean absolute tracking error was approximately 0.04 m/s for both the left and right chain models, with peak errors around 0.28 m/s for the left chain and 0.35 m/s for the right chain. These low errors confirm the model's ability to capture the chain dynamics with high accuracy across the tested operating conditions.

The model quality is assessed using two key metrics: Root mean square error (RMSE) and R^2 (coefficient of determination). The RMSE, which measures the average deviation between model predictions and actual measurements, is calculated as [66]

$$\begin{aligned} RMSE &= \sqrt{\frac{SSR}{n}}, \\ SSR &= \sum_{i=1}^{n} \left(X_{i,Pred} - X_{i,Meas}\right)^2. \end{aligned} \tag{11}$$

In (11), $X_{i,Pred}$ denotes the predicted values, $X_{i,Meas}$ represents the measured values, and n is the number of data points. The sum of squared residuals (SSR) captures the total squared difference between predicted and measured values. The R^2 value quantifies the proportion of variance in the measured data that is explained by the model. It is computed as

$$\begin{aligned} R^2 &= 1 - \frac{SSR}{SST}, \\ SST &= \sum_{i=1}^{n} \left(X_{i,Meas} - \bar{X}_{Meas}\right)^2. \end{aligned} \tag{12}$$

Here, total sum of squares (SST) reflects the total variance in the measurements. An R^2 value closer to 1 indicates that the model can effectively capture the variance in the data, which is ideal for accurate predictions [67].

With R^2 values in percentage of 89.43% for the left chain model and 89.28% for the right chain model, the

models exhibit a strong fit and confirm their reliability for use in trajectory tracking. The combination of high R^2 values demonstrates that the third-order models are effective for the control strategy of the tracked mobile robot.

3.4. Robot kinematics

The tracked mobile robot visualized in Fig. 7 operates using a differential drive system. This system allows the robot to move by controlling the velocities of its two chains independently. The left chain velocity is denoted as v_l, and the right chain velocity is denoted as v_r. The robot's movement in the XY-plane can be described by its linear velocities along the x- and y-axes, and its angular velocity about the z-axis. Here, an instantaneous centres of rotation (ICR) [25,34] is assumed for our tracked robot.

In this study, the robot's kinematic model also assumes a planar surface, where the robot's pose in global coordinates is described by three state variables: its position (x_p, y_p) and its orientation θ_p. The presented kinematic model assumes a fixed instantaneous centre of rotation (ICR) and neglects dynamic ICR shifting caused by track-terrain slip. This assumption holds on flat, high-friction surfaces, but may introduce drift in challenging terrains [2,25]. Future extensions will incorporate slip estimation or online adaptation to capture ICR dynamics for higher fidelity. These variables change as functions of the linear velocity $v(t)$ and angular velocity $\omega(t)$,

$$\begin{aligned} v(t) &= \frac{v_r(t) + v_l(t)}{2}, \\ \omega(t) &= \frac{v_r(t) - v_l(t)}{D}, \end{aligned} \tag{13}$$

as well as the chain separation distance D [5]. The tangential velocity of the wheels is given by the equations $v_r(t) = r\omega_r(t)$ and $v_l(t) = r\omega_l(t)$, where $\omega_r(t)$ and $\omega_l(t)$ are the angular velocities of the chain traction wheels and r is the radius of these wheels including the chain thickness. The kinematic model equations of the robot are given by

$$\begin{bmatrix} \dot{x}_p(t) \\ \dot{y}_p(t) \\ \dot{\theta}_p(t) \end{bmatrix} = \begin{bmatrix} \cos(\theta_p(t)) & 0 \\ \sin(\theta_p(t)) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix}. \tag{14}$$

The integration of the kinematic model, referred as odometry or dead reckoning, gives a robot's pose at sample time T_s. During these time interval velocities $v(t)$ and $\omega(t)$ are assumed to be constant. The assumption of constant velocities during integration steps is justified for small sampling intervals 50-100 ms where velocity changes are minimal, as shown in previous odometry studies [13].

Since $\theta_p(t)$ is not necessarily constant over this interval (it changes due to $\omega(t)$), we approximate it using a Taylor expansion around $t = kT_s$

$$\theta_p(t) \approx \theta_p(k) + \omega(t)(t - kT_s). \tag{15}$$

The exact integration of direct kinematics takes the following form [68]

$$\begin{aligned} \Delta x_p(k) &= v(t) \int_{kT_s}^{(k+1)T_s} \cos(\theta_p(k) + \omega(t)(t - kT_s))dt, \\ \Delta y_p(k) &= v(t) \int_{kT_s}^{(k+1)T_s} \sin(\theta_p(k) + \omega(t)(t - kT_s))dt. \end{aligned} \tag{16}$$

A comparison between measured and modelled data for the left and right chains of our mobile robot is presented in Fig. 8. The robot was manually driven to a target position of $[x_p = 5\text{ m}, y_p = 5\text{ m}]$. The analysis covers the chain velocities, XY-position, orientation and drift for robot's position and orientation, and it highlights the differences in

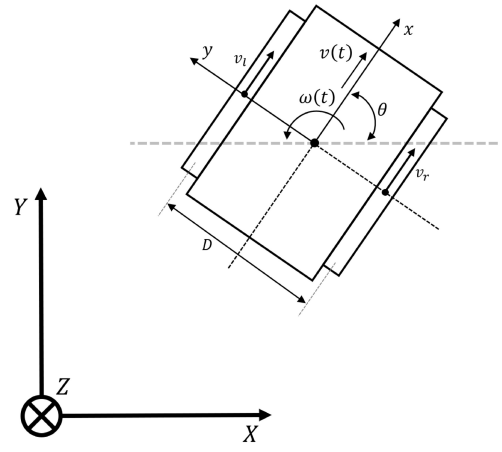


Fig. 7. Kinematic model representation of the differential drive tracked robot navigating on an XY plane.

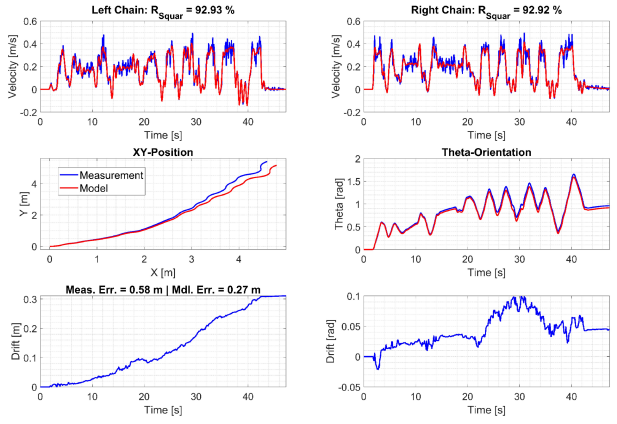


Fig. 8. Comparison of measured and modelled velocity and odometry in teleoperation mode, including position, orientation, and drift analysis for tracked mobile robot.

wheel odometry calculated from measured and modelled chain velocities.

The first row displays the velocity profiles for the left and right chains over time. The left plot illustrates the velocity of the left chain, while the right plot presents the velocity of the right chain. In each case, measured velocities (in blue) are overlaid with the predicted velocities (in red) and provide an assessment of how well the model captures the dynamic behaviour of each chain. The R^2 values, approximately 92.93% for the left chain and 92.92% for the right chain, indicate a strong correlation between measured and predicted velocities and confirm a high degree of model accuracy for velocity tracking. Here, the models from (9) and (10) are used to model the chain dynamics.

The middle row includes the XY-position and theta-orientation plots. The XY-position plot on the left shows the robot's trajectory in the 2D plane, with blue indicating the actual measured trajectory and red representing the predicted trajectory. Although the model tracks the general path, there is a noticeable deviation as the robot moves, particularly as it approaches the target position. The theta-orientation plot on the right compares the measured and modelled orientations (theta) over time. Like the XY-position plot, while the model generally follows the orientation trajectory, deviations arise, especially in regions with rapid orientation changes.

The bottom row provides drift and error analysis. The left plot shows the positional drift over time, calculated as

$$Drift = \sqrt{(x_{meas} - x_{mdl})^2 + (y_{meas} - y_{mdl})^2}. \tag{17}$$

This drift metric quantifies the spatial deviation between the measured and modelled positions throughout the movement. In this measurement, the final measured positional error (indicated in the plot as "Meas. Err.") is 0.58 meters, while the model error (labelled "Mdl. Err.") is 0.27 meters, indicating a growing discrepancy as the robot moves towards the target. The right plot displays the drift in theta-orientation, capturing orientation error over time. Here, deviations accumulate gradually and reflect the difference between measured and modelled orientation.

Overall, the figure illustrates that while the model captures the general motion of the robot, discrepancies accumulate in both position and orientation, particularly in longer trajectories. This accumulation results from the assumption of constant velocities in (16), numerical integration drift, and unmodelled effects such as slippage or terrain variations. These factors cause small errors to propagate over time and lead to increasing deviations in the predicted trajectory.

3.5. Nonlinear state-space modelling

To derive the closed-loop state-space model for the left chain system combined with the discrete PI controller, we need to combine both systems into a single augmented system. This involves using the output of the PI controller (which is the PWM signal) as the input to the left chain system and using the difference between the set velocity and actual velocity as the input to the PI controller. A similar process is performed for the right chain to obtain its closed-loop state-space model.

The state-space model of the left chain (with PWM as the input and actual chain velocity as the output) is given by

$$\begin{aligned} x_{k+1} &= A_l x_k + B_l u_k, \\ y_k &= C_l x_k, \end{aligned} \tag{18}$$

where

1) x_k is the state vector for the left chain system,
2) u_k is the PWM input (output from the PI controller),
3) y_k is the velocity output (measured velocity).

The discrete state-space model of the left PI controller is provided as [69]

$$\begin{aligned} x_{PI,k+1} &= A_{PI} x_{PI,k} + B_{PI} e_k, \\ u_k &= C_{PI} x_{PI,k} + D_{PI} e_k, \end{aligned} \tag{19}$$

where

1) $e_k = v_{set} - y_k$ is the velocity error,
2) x_{PI} is the evolution of the integral of the error,
3) u_k is the PWM output of the PI controller,
4) $A_{PI} = 1$, $B_{PI} = T_s$, $C_{PI} = K_i$, $D_{PI} = K_p$ are the PI controller matrices.

For combining both systems, we consider the input to the PI controller is the error $e_k = v_{set} - y_k$. Since $y_k = C_l x_k$, the error can be written as

$$e_k = v_{set} - C_l x_k. \tag{20}$$

The PWM input to the left chain system is the output of the PI controller

$$u_k = C_{PI} x_{PI,k} + D_{PI} e_k. \tag{21}$$

Substitute the expression for e_k into this equation

$$u_k = C_{PI} x_{PI,k} + D_P I(v_{set} - C_l x_k). \tag{22}$$

Now, we can substitute u_k into the state equation for the left chain system

$$x_{k+1} = A_l x_k + B_l(C_{PI} x_{PI,k} + D_{PI}(v_{set} - C_l x_k)). \tag{23}$$

This gives the closed-loop state equation for the left chain system combined with the PI controller.

To build the augmented state-space model, we define the augmented state vector

$$\boldsymbol{X}_k = \begin{bmatrix} x_k \\ x_{PI,k} \end{bmatrix}. \tag{24}$$

Now, we write the augmented state-space equations

$$\boldsymbol{X}_{k+1} = \begin{bmatrix} A_l - B_l D_{PI} C_l & B_l C_{PI} \\ -B_{PI} C_l & A_{PI} \end{bmatrix} \boldsymbol{X}_k + \begin{bmatrix} B_l D_{PI} \\ B_{PI} \end{bmatrix} v_{set}. \tag{25}$$

And the output equation

$$y_k = \begin{bmatrix} C_l & 0 \end{bmatrix} \boldsymbol{X}_k. \tag{26}$$

Given the robot's left wheel velocity v_l and right wheel velocity v_r, the nonlinear kinematic equations governing the motion of the robot in terms of its global position x, y, and orientation θ (heading angle) were introduced in (14).

The state vector for the nonlinear system would be

$$\boldsymbol{x}_{nonlinear} = \begin{bmatrix} x_p \\ y_p \\ \theta_p \end{bmatrix}. \tag{27}$$

The input vector for the nonlinear kinematics model depends on the left and right chain velocities, which are controlled by the motor PWM signals u_l and u_r. However, in this case, v_l and v_r are considered as inputs

$$\boldsymbol{u} = \begin{bmatrix} v_l \\ v_r \end{bmatrix}. \tag{28}$$

The total state vector $\boldsymbol{x}$ will include the augmented states of both chains (x_l and x_r) from (25) and the robot's position and orientation (x, y, and θ) from (14)

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{x}_l \\ \boldsymbol{x}_r \\ \boldsymbol{x}_p \\ \boldsymbol{y}_p \\ \boldsymbol{\theta}_p \end{bmatrix} \in \mathbb{R}^{11}, \tag{29}$$

where

1) $x_l \in \mathbb{R}^4$ is the left chain state vector,
2) $x_r \in \mathbb{R}^4$ is the right chain state vector,
3) x_p, y_p, $\theta_p \in \mathbb{R}$ is the odometry state vector.

The input vector $\boldsymbol{u}$ of the combined state space mode consists of the reference velocities for the left and right chains

$$\boldsymbol{u} = \begin{bmatrix} \boldsymbol{v}_{ref,l} \\ \boldsymbol{v}_{ref,r} \end{bmatrix}. \tag{30}$$

The output vector is the odometry states (x, y, and θ) and the chain velocities v_l and v_r.

Finally, the full discrete-time nonlinear state-space model $\boldsymbol{x}_{k+1} = \boldsymbol{f}(\boldsymbol{x}_k, \boldsymbol{u}_k)$ can be summarized as

$$\begin{aligned} \boldsymbol{x}_{l,k+1} &= \boldsymbol{A}_l \boldsymbol{x}_{l,k} + \boldsymbol{B}_l \boldsymbol{u}_{l,k}, \\ \boldsymbol{x}_{r,k+1} &= \boldsymbol{A}_r \boldsymbol{x}_{r,k} + \boldsymbol{B}_r \boldsymbol{u}_{r,k}, \\ \boldsymbol{x}_{p,k+1} &= \boldsymbol{x}_{p,k} + \boldsymbol{T}_s \boldsymbol{v}_k \operatorname{sinc}\left(\frac{\boldsymbol{\omega}_k \boldsymbol{T}_s}{2}\right) \cos\left(\boldsymbol{\theta}_k + \frac{\boldsymbol{\omega}_k \boldsymbol{T}_s}{2}\right), \\ \boldsymbol{y}_{p,k+1} &= \boldsymbol{y}_{p,k} + \boldsymbol{T}_s \boldsymbol{v}_k \operatorname{sinc}\left(\frac{\boldsymbol{\omega}_k \boldsymbol{T}_s}{2}\right) \sin\left(\boldsymbol{\theta}_k + \frac{\boldsymbol{\omega}_k \boldsymbol{T}_s}{2}\right), \\ \boldsymbol{\theta}_{p,k+1} &= \theta_{p,k} + \boldsymbol{\omega}_k \boldsymbol{T}_s, \end{aligned} \tag{31}$$

where the last three states come from integrating the continuous-time differential-drive kinematics in (15) and (16) over one sampling interval (with constant track speeds during that interval). The translation and rotation speeds of the robot are resulted as $\boldsymbol{v}_k = \frac{\boldsymbol{C}_l \boldsymbol{x}_{l,k} + \boldsymbol{C}_r \boldsymbol{x}_{r,k}}{2}$ and $\boldsymbol{\omega}_k = \frac{\boldsymbol{C}_r \boldsymbol{x}_{r,k} - \boldsymbol{C}_l \boldsymbol{x}_{l,k}}{\boldsymbol{D}}$.

The nonlinear state-space representation in (31) can serve as the foundation for a nonlinear model predictive control (NMPC) framework. It can be employed to predict the system's future behaviour over the prediction horizon and to optimize the control inputs, $v_{ref,l}$ and $v_{ref,r}$, for achieving desired tracking performance.

While higher-order nonlinear models (e.g., quadratic in state/control) can increase fidelity, they introduce additional computational complexity [17], which conflicts with real-time constraints on embedded hardware. Our identified third-order linear dynamics combined with nonlinear kinematics provide a practical trade-off, but future work may explore higher-fidelity models if hardware permits.

4. RESULTS AND DISCUSSION

In this section, we present experimental results confirming that our approach fulfils the main contributions of the paper.

4.1. Augmented chain-dynamics state-space model

Fig. 9 demonstrates the ability of PI controllers to track the reference velocities of the left and right chains of our tracked mobile robot according to the identified state-space representations from (9) and (10). In the case of both graphs the main characteristic being compared is the reference velocity (blue line) against the actual velocity (red

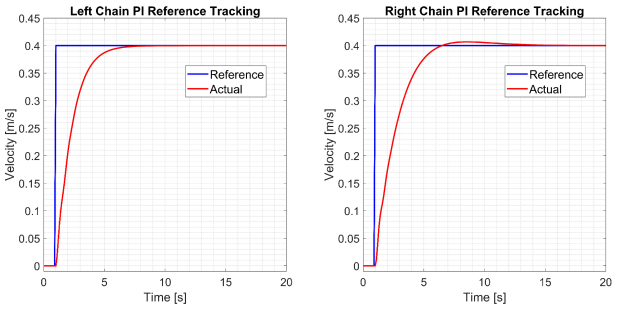


Fig. 9. Performance of PI controllers in tracking reference velocities for left and right chains.

Table 2. Comparison of PI controller parameters and performance metrics for left and right chains.

		Left chain	Right chain
Parameters	**Proportional**	0.0931	0.0749
	Integral	1.8638	1.4984
Performance and robustness	**Rise time**	2.4 s	3.1 s
	Settling time	4.2 s	4.6 s
	Overshoot	0%	1.98%
	Closed-loop stability	stable	stable

line) with time on the horizontal axis. For the left chain, the PI controller provides the rise time of about 2.4 sec and settling time of about 4.2 sec, while there is no overshoot occurred. On the other hand, the right chain with a rise time of 3.1 s, settling time of 4.6 s and small overshoot of 1.98%. These differences result from the difference in the left and right chains, most probably due to the differences in the physical parameters such as the friction parameters and the load distributions. These differences in the chain dynamic were observed in the identified and selected state-space representations in (9) and (10).

The PI parameters of both chains are identified by particle swarm optimization and guarantee that each of the chains had stable and optimal closed-loop behaviour. PSO was configured with 30 particles, a maximum of 500 iterations, inertia weight linearly decreasing from 0.9 to 0.4, and cognitive/social factors set to 2. The optimization minimized rise time, settling time, overshoot, and steady-state error in a weighted cost function. Convergence was typically achieved within 200 iterations. Table 2 shows that the PI parameters for the left chain set a higher response rate without overshoot, while those for the right chain set a slightly slower response but with minimal overshoot. This explains why the tuning is asymmetric, because the PSO process is inherently flexible in achieving the best control for each chain based on its dynamic performance. In our objective function of PSO, it was clearly defined to penalize discrepancies more than 25% in rise and settling time.

Finally, the optimized values of rise and settling time were compared. During each iteration of the PSO, we also included a penalty factor that guarantees the stability condition in our control loop. This is achieved by ensuring that all eigenvalues of the discrete-time closed-loop system matrix comply with the condition specified in (32) [70]

$$|\lambda_i| < 1, \ \forall i, \tag{32}$$

where λ_i are the eigenvalues of the discrete-time closed-loop system matrix.

The following are the state-space models for the closed-loop system with incorporation of the chain dynamics and the PI controllers for both left and right chains. These models are derived from the linear discrete-time representation of the chain dynamics, as described by (33) and (34), and the PI controller parameters in Table 2.

For the left chain, the closed-loop discrete-time state-space model is given by

$$\begin{aligned} x_{l,k+1} &= \begin{bmatrix} -0.0153 & 1 & 0 & 0.3066 \\ -0.0147 & 0 & 1 & 0.2939 \\ 0.4754 & -1.3636 & 1.7229 & 0.1356 \\ -0.1 & 0 & 0 & 1 \end{bmatrix} x_{l,k} \\ &\quad + \begin{bmatrix} 0.0153 \\ 0.0147 \\ 0.0068 \\ 0.1 \end{bmatrix} u_{l,k}, \\ y_{l,k} &= \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} x_(l,k), \end{aligned} \tag{33}$$

where

1) $u_{l,k}$ is the left chain reference velocity,
2) $y_{l,k}$ is the left chain actual velocity.

For the right chain, the state-space model is given by (34), where

1) $u_{r,k}$ is the right chain reference velocity,
2) $y_{r,k}$ is the right chain actual velocity.

$$\begin{aligned} x_{r,k+1} &= \begin{bmatrix} -0.0099 & 1 & 0 & 0.1972 \\ -0.0169 & 0 & 1 & 0.3389 \\ 0.4877 & -1.2406 & 1.7163 & 0.1583 \\ -0.1 & 0 & 0 & 1 \end{bmatrix} x_{r,k} \\ &\quad + \begin{bmatrix} 0.0099 \\ 0.0169 \\ 0.0079 \\ 0.1 \end{bmatrix} u_{r,k}, \\ y_{r,k} &= \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} x_{r,k}. \end{aligned} \tag{34}$$

These state-space models encapsulate the combined dynamics of the chain and PI controllers and provide a foundation for analysing the closed-loop behaviour of each chain in response to reference velocity inputs.

4.2. Real-time feasibility analysis

The effectiveness of the left and right chain state-space models of our chain-based mobile robot in terms of the mean runtime for 10000 runs per prediction step is shown in Fig. 10. The runtime analysis was carried out on a laptop with Intel(R) Xeon(R) E-2176M CPU @ 2.70 GHz. The x-axis is labelled as the number of prediction steps and y-axis indicates the mean runtime in microseconds. As shown, runtimes increase with the number of prediction steps and reach approximately 13 μs at 40 steps for both chains. A notable data point is marked at 10 prediction steps, where the mean runtime is approximately 4.08 μs

for the left and right chains. This specific prediction step configuration was selected based on computational feasibility. The runtime analysis shows that 10 steps result in a mean runtime that makes real-time execution feasible on embedded hardware such as the Raspberry Pi 4 Model B. Increasing the prediction horizon significantly raises computational costs, which can exceed real-time processing limits.

The runtime peaks observed at prediction steps 14 and 16 during the state-space model calculations for the left and right chains can be attributed to numerical inefficiencies and computational overhead associated with floating-point operations and matrix manipulations. These peaks are a likely due to a combination of factors and include increased data dependencies and cache memory access patterns that occur at specific step sizes. When performing repetitive calculations over an increasing number of prediction steps, the data structures used for storing intermediate results can exceed the CPU cache size and cause the processor to fetch data from slower main memory. This phenomenon, commonly referred to as "cache thrashing," leads to temporary spikes in computational time for certain step configurations, as discussed in [71]. Furthermore, the matrix manipulations at this number of prediction steps may involve a higher number of non-zero elements or condition numbers that temporarily increase computational load. This is particularly relevant in state-space calculations, where the structure of the matrices and the operations performed on them (e.g., multiplications or summations) can lead to nonlinear variations in runtime. These effects have also been highlighted in works such as [72], which discuss the computational cost of matrix operations in iterative processes. Lastly, slight variations in runtime can also result from system-level interrupts or background processes that interfere with the uniform execution of computational tasks, especially on multi-threaded CPUs like the Intel Xeon used in this study.

For deployment on real-time hardware, specifically the Raspberry Pi 4 Model B, we estimate the runtime by considering the difference in processing power between the test hardware (Intel Xeon E-2176M) and the target hardware. A common approach is to approximate the runtime

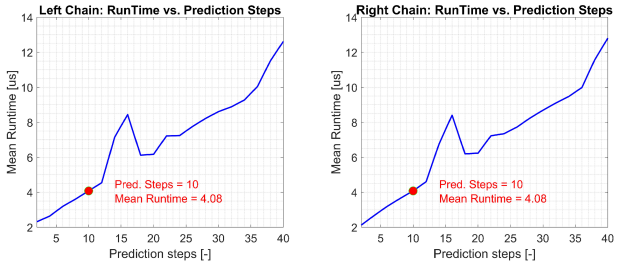


Fig. 10. Mean runtime per prediction step for left and right chain state-space models on Intel Xeon processor.

based on CPU benchmark scores, floating point operations per second (FLOPs), or clock speed differences. Assuming linear scalability, the estimated runtime on the Raspberry Pi T_{Rpi4} can be computed as follows:

$$T_{Rpi4} = T_{Xeon} \times Ratio, \tag{35}$$

where the *Ratio* is determined using three different values

Clock speed ratio: The relative difference in GHz between the two CPUs.

Benchmark score ratio: A normalized comparison of CPU performance based on industry benchmarks.

FLOP-based ratio: A measure of floating-point computation performance.

Table 3 summarizes the estimated runtime values on the Raspberry Pi 4 Model B for 10 prediction steps based on the three different ratios.

This suggests that, depending on the estimation method, the runtime for 10 prediction steps on the Raspberry Pi 4 Model B will range between approximately 7.3 to 78.3 μs. These values indicate feasibility for real-time applications in model predictive control and ensure that computational efficiency is maintained within practical limits.

To validate these estimates, an augmented state-space model for both the left and right chains was implemented in Simulink and deployed as a ROS2 node on the Raspberry Pi 4 Model B. The execution time of the C++ node function was measured using the chrono library over a duration of 2 minutes, with execution times recorded in a log file. The results were then analysed and plotted in MATLAB as shown in Fig. 11.

The experimental results indicate an average execution time of 77.37 μs with a standard deviation of 5.18 μs. The execution time distribution follows a Gaussian pattern and confirms consistency in the measurements.

The experimental runtime of 77.37 μs aligns closely with the estimate derived from FLOP-based scaling (78.336 μs). This confirms that using FLOP-based performance is a reliable approach for estimating execution

Table 3. Estimated runtime on Raspberry Pi 4 Model B for 10 prediction steps using different scaling methods.

Method	**Ratio** (Xeon to RPi4)	**Estimated runtime** (us)
Clock speed (2.7 GHz / 1.5 GHz)	1.8	7.344
Benchmark score (4770/617) [73,74]	7.731	31.542
FLOPs performance (259.2 GFLOPS / 13.5 GFLOPS) [75,76]	19.2	78.336

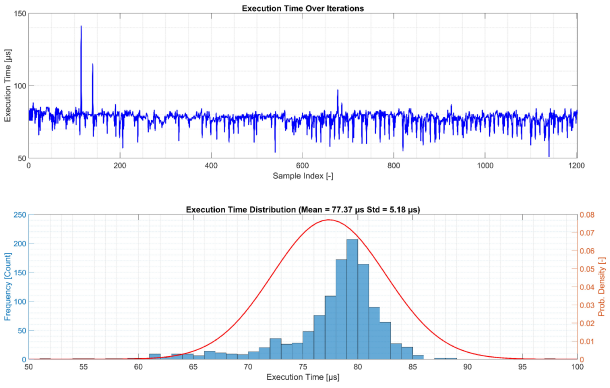


Fig. 11. Experimental execution time analysis on Raspberry Pi 4 Model B.

time on hardware with different computational capabilities. However, the estimates based on clock speed (7.344 μs) and benchmark scores (31.542 μs) significantly underestimate the actual execution time. This discrepancy highlights that pure clock speed or benchmark-based scaling does not fully capture the complexity of execution, particularly for floating-point intensive computations in state-space model evaluations.

It is quite important to choose an appropriate number of prediction steps for achieving reasonable levels of performance and computational time in MPC frameworks. Previous works on predictive control for mobile robots [77,78] suggest that a prediction horizon of 10 steps is appropriate since more steps would require significant computational resources. On the other hand, fewer steps would yield low control quality due to the short prediction horizon. This setting provides enough prediction accuracy and, at the same time, is deployable in real time on limited hardware such as Raspberry Pi.

To estimate the real-time runtime for implementing model predictive control (MPC) on a Raspberry Pi 4 Model B, we consider the requirements for calculating two state-space models (one for each chain) with a prediction horizon of 10 steps over 10 iterations. Previous research suggests that 5 to 10 iterations are typically sufficient to achieve convergence in real-time MPC applications and balance computational feasibility with control accuracy [79,80]. Given the mean runtime of approximately 77.37 μs per 10 prediction steps for a single model on the Raspberry Pi 4 Model B (as derived from the benchmark on the Intel Xeon processor), the runtime for two state-space models with 10 prediction steps and 10 iterations would be

$$T_{Rpi4,MPC} \approx 10 \times 77.37\ \mu\text{s} = 773.7\ \mu\text{s}. \tag{36}$$

This total runtime of approximately 773.7 μs is well within the real-time requirement for a discrete time step of $T_s = 0.1$ s. This establishes the fact that an MPC approach with 10 steps of prediction and up to 10 iterations can be implemented in Raspberry Pi 4 Model B hardware within reasonable computational efficiency while offering required control performance that is satisfactory for real-time control applications.

While our analysis demonstrates that the forward model can be evaluated in ~ 77 μs on the Raspberry Pi 4, for full MPC feasibility, it is essential to account for solver computation and actuator command latency. Based on literature [15,16], typical solver times for low-dimensional NMPC with horizons of 10-20 steps are within 1-3 ms on embedded processors. Combined with our model evaluation time and the $\sim$ 1-2 ms actuator command latency, we anticipate an overall control loop execution within 5 ms, compatible with 200 Hz updates. Future work will experimentally validate full closed-loop MPC timing.

5. CONCLUSION

In this work, we presented a systematic approach to modeling and identification for a tracked mobile robot that emphasizes both accuracy and real-time feasibility. Our method combines particle swarm optimization and information-theoretic criteria to identify compact state-space models that capture the complex dynamics of tracked locomotion. We demonstrated that this approach yields high-fidelity models (achieving over 92% variance explained in validation tests) while avoiding unnecessary complexity. We also integrated the identified chain dynamics with the vehicle's kinematics to form a nonlinear state-space model of the robot, which was shown to predict the robot's motion with excellent accuracy in real-world trials. Furthermore, by conducting a detailed runtime analysis on embedded hardware, we verified that the resulting model can be used for real-time model predictive control without exceeding computational limits. To the best of our knowledge, this work is one of the first to successfully bridge advanced system identification techniques with practical real-time control deployment for a tracked robot.

Our findings have several implications. First, the success of PSO in tuning both the model parameters and controller gains suggests that global optimization methods are a robust tool for system identification in robotics, especially when paired with model selection metrics to guard against overfitting. Second, the accurate tracking of the robot's trajectory using the identified model underscores the importance of including detailed dynamics (like track friction and drive response) in mobile robot models - even moderate improvements in model fidelity can translate to significantly better control performance. Third, the real-time validation indicates that even relatively complex models can be executed on low-cost hardware if one carefully controls the model order and optimizes the implementation. This opens opportunities to deploy more so-

phisticated controllers (like MPC) on platforms that were previously considered too resource-constrained.

Future work will focus on leveraging the developed model within a full MPC-based autonomous navigation system. In particular, we plan to implement MPC for trajectory tracking on the Horizon XIX rover and evaluate its performance in challenging scenarios (e.g., aggressive maneuvers on irregular terrain). The model may be further refined by incorporating adaptive elements - for instance, on-line identification of track slip parameters [2,25] - to maintain accuracy as environmental conditions change. While the current model simplifies track-terrain interactions, our future work will address dynamic ICR behaviour and slip modelling for rough terrain applications. Another avenue is to integrate additional dynamics such as payload effects or high-frequency track vibrations, should they become significant for specific tasks. Finally, while our current experiments were conducted at relatively low speeds, we intend to test the approach at higher speeds and on different ground surfaces to validate its robustness. We believe that the approach presented here provides a strong foundation for advanced control of tracked robots, and that with these continued improvements, it can greatly enhance the autonomy and reliability of such systems in real-world operations.

DECLARATIONS

Conflict of Interest

The authors declare that they have no conflict of interest.

Authors' Contributions

Hamza Hobbani was responsible for the conception, implementation, experimentation, and writing of the manuscript. Fernando Perez-Peña and Karsten Schmidt provided scientific supervision, critical feedback, and manuscript revisions. All authors reviewed and approved the final version of the manuscript.

Funding

The work described in this paper was funded and supported by the Department of Computing and Engineering of the Frankfurt University of Applied Sciences and the Department of Automation, Electronics and Computer Architecture and Networks of the University of Cadiz.

REFERENCES

[1] *Springer Handbook of Robotics*, Springer, Berlin, Heidelberg, 2008.

[2] X. Zhao, E. Lu, Z. Tang, C. Luo, L. Xu, and H. Wang, "Trajectory prediction method for agricultural tracked robots based on slip parameter estimation," *Computers and Electronics in Agriculture*, vol. 222, 109057, 2024.

[3] H. R. Everett, *Sensors for Mobile Robots: Theory and Application*, Peters, Wellesley, Massachusetts, 2001.

[4] R. Murphy, *Disaster Robotics*, The MIT Press, Cambridge, Massachusetts, 2017.

[5] A. Amangeldi, A. Dospanova, and S. Kusdavletov, "Multi-goal point stabilization control for differential drive mobile robot with odometry," *Proc. of 15th International Conference on Computer and Automation Engineering (ICCAE)*, Sydney, Australia, pp. 389-393, 2023.

[6] W. Farag, "Complex trajectory tracking using PID control for autonomous driving," *International Journal of Intelligent Transportation Systems Research*, vol. 18, no. 2, pp. 356-366, 2020.

[7] N. H. Thai, T. T. K. Ly, H. Thien, and Q. L. Dzung, "Trajectory tracking control for differential-drive mobile robot by a variable parameter PID controller," *International Journal of Mechanical Engineering and Robotics Reserch*, vol. 11, no. 8, pp. 614-621, 2022.

[8] U. Zangina, S. Buyamin, M. S. Z. Abidin, M. S. A. Mahmud, and H. S. Hasan, "Non-linear PID controller for trajectory tracking of a differential drive mobile robot," *Journal of Mechanical Engineering Research and Developments*, vol. 43, no. 7, pp. 255-270, 2020. [Online]. Available: http://eprints.utm.my/90651/1/umarzangina2020_nonlinearpidcontrollerfortrajectorytracking.pdf

[9] T. N. T. Cao, B. T. Pham, H. D. Tran, L. X. Gia, N. T. Nguyen, and V. N. Truong, "Non-singular terminal sliding mode control for trajectory-tracking of a differential drive robot," *Proc. of E3S Web of Conferneces*, vol. 496, 2005, 2024.

[10] C.-Y. Chen, T.-H. S. Li, Y.-C. Yeh, and C.-C. Chang, "Design and implementation of an adaptive sliding-mode dynamic controller for wheeled mobile robots," *Mechatronics*, vol. 19, no. 2, pp. 156-166, 2009,.

[11] A. Filipescu, V. Minzu, B. Dumitrascu, A. Filipescu, and E. Minca, "Trajectory-tracking and discrete-time sliding-mode control of wheeled mobile robots," *Proc. of IEEE International Conference on Information and Automation*, Shenzhen, China, pp. 27-32, 2011.

[12] F. Kühne, W. F. Lages, and J. M. G. da Silva Jr., "Model predictive control of a mobile robot using linearization," *Proceedings of Mechatronics and Robotics*, vol. 4, no. 4, pp. 525-530, 2004. [Online]. Available: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=634a548d992534d769b5b8597c796c6b1153f628

[13] Y. Tange, C. Nakazawa, and T. Matsui, "A multi-variable disturbance observer for model predictive control," *Proc. of 17th Mediterranean Conference on Control and Automation*, Thessaloniki, Greece, pp. 856-861, 2009.

[14] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*, Cambridge University Press, Cambridge, 2017.

[15] S. Yu, M. Hirche, Y. Huang, H. Chen, and F. Allgöwer, "Model predictive control for autonomous ground vehicles: A review," *Autonomous Intelligent Systems*, vol. 1, no. 1, 2021.

[16] M. N. Zeilinger, D. M. Raimondo, A. Domahidi, M. Morari, and C. N. Jones, "On real-time robust model predictive control," *Automatica*, vol. 50, no. 3, pp. 683-694, 2014.

[17] C. Conte, T. Summers, M. N. Zeilinger, M. Morari, and C. N. Jones, "Computational aspects of distributed optimization in model predictive control," *Proc. of IEEE 51st IEEE Conference on Decision and Control (CDC)*, Maui, HI, USA, pp. 6819-6824, 2012.

[18] A. Grancharova, T. A. Johansen, and P. Tøndel, "Computational aspects of approximate explicit nonlinear model predictive control," *Lecture Notes in Control and Information Sciences, Assessment and Future Directions of Nonlinear Model Predictive Control*, R. Findeisen, F. Allgöwer, and L. T. Biegler, Eds., Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 181-192, 2007.

[19] M. B. Saltık, L. Özkan, J. H. Ludlage, S. Weiland, and P. M. van den Hof, "An outlook on robust model predictive control algorithms: Reflections on performance and computational aspects," *Journal of Process Control*, vol. 61, pp. 77-102, 2018.

[20] J. L. Martinez, A. Mandow, J. Morales, A. Garcia-Cerezo, and S. Pedraza, "Kinematic modelling of tracked vehicles by experimental identification," *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1487-1492, 2004.

[21] R. González, F. Rodríguez, and J. L. Guzmán, "Modelling tracked robots in planar off-road conditions," in *Autonomous Tracked Robots in Planar Off-Road Conditions. Studies in Systems, Decision and Control*, vol 6. Springer, Cham, pp. 11-33, 2014.

[22] L. Bruzzone, S. E. Nodehi, and P. Fanghella, "Tracked locomotion systems for ground mobile robots: A review," *Machines*, vol. 10, no. 8, 648, 2022.

[23] A. Pappalettera, G. Reina, and G. Mantriota, "Design and analysis of tracked stair-climbing robot using innovative suspension system," *Robotics*, vol. 13, no. 3, 45, 2024.

[24] A. J. Dean and S. N. Brennan, "A low-order dynamic model of a tracked robot inclusive of non-linear slip," *International Journal of Heavy Vehicle Systems (IJHVS)*, vol. 15, no. 2/3/4, 327, 2008.

[25] H. Lu, G. Xiong, and K. Guo, "Motion predicting of autonomous tracked vehicles with online slip model identification," *Mathematical Problems in Engineering*, vol. 2016, pp. 1-13, 2016.

[26] R. Li, X. Zhang, S. Hu, J. Wu, Y. Feng, and Y. Yao, "Design and analysis of an adaptive obstacle-overcoming tracked robot with passive swing arms," *Machines*, vol. 11, no. 12, 1051, 2023.

[27] M. Ahmad, V. Polotski, and R. Hurteau, "Path tracking control of tracked vehicles," *Proc. of IEEE International Conference on Robotics and Automation*, pp. 2938-2943, 2000.

[28] C. Jin, C. Wang, K. Tang, C. Jiao, and Y. Wang, "Research and application of a coupled wheel-track off-road robot based on separate track structure," *Advances in Mechanical Engineering*, vol. 16, no. 9, 2024.

[29] X. Han, M. Lin, X. Wu, and J. Yang, "Design of an articulated-tracked mobile robot with two swing arms," *Proc. of IEEE 4th International Conference on Advanced Robotics and Mechatronics (ICARM)*, pp. 684-689, 2019.

[30] I. K. Romanova and A. A. Shchurikhin, "Problems of using a tracked robot in group control of heterogeneous robots," *Proc. of International Conference on Informatics, Technology, and Engineering*, 2021.

[31] Y. Cao, B. Liu, and J. Pu, "Robust control for a tracked mobile robot based on a finite-time convergence zeroing neural network," *Frontiers in Neurorobotics*, vol. 17, 1242063, 2023.

[32] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359-366, 1989.

[33] M. J. Almashaal, A. R. Gaiduk, and S. G. Kapustyan, "Tracked robot motion control system," in Kravets, A.G., Bolshakov, A.A., Shcherbakov, M.V. (eds) *Cyber-Physical Systems Engineering and Control. Studies in Systems, Decision and Control*, vol 477. Springer, Cham, pp. 87-97, 2023.

[34] J. L. Martínez, A. Mandow, J. Morales, S. Pedraza, and A. García-Cerezo, "Approximating kinematics for tracked mobile robots," *The International Journal of Robotics Research*, vol. 24, no. 10, pp. 867-878, 2005.

[35] S. A. Pauliukavets, A. A. Velchenko, A. A. Radkevich, and D. Y. Tschaplygin, "Modeling of control system for tracked mobile robot taking into account kinematic and dynamic parameters," *Science & Technique*, vol. 23, no. 1, pp. 5-14, 2024.

[36] Z. Zhao, H. Liu, H. Chen, J. Hu, and H. Guo, "Kinematics-aware model predictive control for autonomous high-speed tracked vehicles under the off-road conditions," *Mechanical Systems and Signal Processing*, vol. 123, pp. 333-350, 2019.

[37] Z. Ziye, L. Haiou, C. Huiyan, X. Shaohang, and L. Wenli, "Tracking control of unmanned tracked vehicle in off-road conditions with large curvature," *Proc. of IEEE Intelligent Transportation Systems Conference (ITSC)*, pp. 3867-3873, 2019.

[38] A. Ugenti, R. Galati, G. Mantriota, and G. Reina, "Analysis of an all-terrain tracked robot with innovative suspension system," *Mechanism and Machine Theory*, vol. 182, 105237, 2023.

[39] S. M. Shafaei and H. Mousazadeh, "Motion energy perspective of tracked locomotion system of autonomous tractor-trailer robot," *Energy*, vol. 264, 126520, 2023.

[40] Y. Zhang, Z. Cheng, J. Wang, L. Wang, and F. Sun, "Structural design and motion characteristics analysis of dual-mode active swing arm tracked robot," *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, vol. 239, no. 4, 2024.

[41] G. He, J. Shi, C. Liu, W. Guo, and X. Sheng, "Autonomous navigation of tracked robot in uneven terrains," In: Yang, H., *et al. Intelligent Robotics and Applications. ICIRA 2023. Lecture Notes in Computer Science*, vol. 14274. Springer, Singapore, pp. 74-84, 2023.

[42] S.-M. Petrişor and G. Bârsan, "Aspects on the design, implementation, and simulation of a tracked mini robot destined for special applications in theatres of operations," *Proc. of Sixth International Conference on Machine Vision (ICMV 2013)*, London, United Kingdom, 2013.

[43] C. Wang, H. Zhang, H. Ma, S. Wang, X. Xue, H. Tian, and P. Liu, "Simulation and validation of a steering control strategy for tracked robots," *Applied Sciences*, vol. 13, no. 19, 11054, 2023.

[44] C. Wangm, S. Wang, H. Ma, H. Zhang, X. Xue, H. Tian, and L. Zhang, "Research on the obstacle-avoidance steering control strategy of tracked inspection robots," *Applied Sciences*, vol. 12, no. 20, 10526, 2022, doi: 10.3390/app122010526.

[45] C. Wang, H. Ma, X. Xue, Q. Mao, J. Song, R. Wang, and Q. Liu, "Research on the deviation correction control of a tracked drilling and anchoring robot in a tunnel environment," *Actuators*, vol. 13, no. 6, 221, 2024.

[46] R. Xu and C. Liu, "Tracked robot with underactuated tension-driven RRP transformable mechanism: Ideas and design," *Frontiers of Mechanical Engineering*, vol. 19, no. 1, 2024.

[47] V. Korendiy, O. Kachur, M. Boikiv, Y. Novitskyi, and O. Yaniv, "Analysis of kinematic characteristics of a mobile caterpillar robot with a SCARA-type manipulator," *Transport Technologies*, vol. 2023, no. 2, pp. 56-67, 2023.

[48] J. Zhao, J. Zhang, H. Liu, J. Wang, and Z. Chen, "Path planning for a tracked robot traversing uneven terrains based on tip-over stability," *Asian Journal of Control*, vol. 25, no. 5, pp. 3569-3583, 2023.

[49] C. Zong, Z. Ji, and H. Yu, "Dynamic stability analysis of a tracked mobile robot based on human-robot interaction," *Assembly Automation*, vol. 40, no. 1, pp. 143-154, 2019.

[50] J. Wang, C. Liu, W. Wei, and Q. Yan, "Design and Optimization of a Reconfigurable Wheel-Tracked Mobile Robot," *Mechatronics and Automation Technology*, vol. 46, pp. 155-165, 2023.

[51] E. Lu, Z. Zhao, J. Yin, C. Luo, and Z. Tian, "Trajectory learning and reproduction for tracked robot based on bagging-GMM/HSMM," *Journal of Electrical Engineering & Technology*, vol. 18, no. 6, pp. 4441-4453, 2023.

[52] H. Pan, X. Chen, J. Ren, B. Chen, K. Huang, H. Zhang, and H. Lu, "Deep reinforcement learning for flipper control of tracked robots in urban rescuing environments," *Remote Sensing*, vol. 15, no. 18, 4616, 2023.

[53] S. Erchen, M. Triller, L. Sohlbach, and K. Schmidt, "Development of a chain-based mobile robot," *Proc. of 21st International Conference on Research and Education in Mechatronics (REM)*, Cracow, Poland, pp. 1-4, 2020.

[54] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.

[55] K. Smerpitak, W. Jearnpanitpong, A. Julsereewong, and T. Thepmanee, "Multi-PLC control system based on wireless bridge/base stations for work-in-process movements in corrugated box manufacturer," *Proc. of International Conference on Control, Automation and Systems (ICCAS)*, 2018.

[56] V. Ancheva and V. Voinohovska, "Minicomputer Rasberry PI 4 as an effective instrument in conduction STEM education in informatics classes," *Proc. of ICERI2020 Online Conference*, pp. 2519-2524, 2020.

[57] G. Ciaccio, M. Ehlert, and B. Schnor, "Exploiting gigabit ethernet capacity for cluster applications," *Proc. of 27th Annual IEEE Conference on Local Computer Networks*, Tampa, FL, USA, pp. 669-678, 2002.

[58] J. Kennedy and R. Eberhart, "Particle swarm optimization," *Proc. of ICNN'95 - International Conference on Neural Networks*, Perth, WA, Australia, pp. 1942-1948, 1995.

[59] R. Eberhart and J. Kennedy, "A new optimizer using particle swarm theory," *Proc. of the Sixth International Symposium on Micro Machine and Human Science*, Nagoya, Japan, pp. 39-43, 1995.

[60] T. Huang and A. S. Mohan, "A hybrid boundary condition for robust particle swarm optimization," *IEEE Antennas and Wireless Propagation Letters*, vol. 4, pp. 112-117, 2005.

[61] K. P. Burnham and D. R. Anderson, *Model Selection and Multimodel Inference: A Practical Information-theoretic Approach*, 2nd ed., Springer, New York, 2010.

[62] H. Akaike, *Information Theory and an Extension of the Maximum Likelihood Principle*, Springer Series in Statistics, Selected Papers of Hirotugu Akaike, E. Parzen, K. Tanabe, and G. Kitagawa, Eds., Springer New York, New York, NY, pp. 199-213, 1998.

[63] G. Schwarz, "Estimating the dimension of a model," *Annals of Statistics*, vol. 6, no. 2, pp. 461-464, 1978.

[64] R. Poli, J. Kennedy, and T. Blackwell, "Particle swarm optimization," *Swarm Intelligence*, vol. 1, no. 1, pp. 33-57, 2007.

[65] T. Kailath, *Linear Systems*, Prentice-Hall, Englewood Cliffs, N.J., 1980.

[66] N. R. Draper and H. Smith, *Applied Regression Analysis*, 3rd ed., John Wiley & Sons, New York, 1998.

[67] A. Simpkins, "System identification: Theory for the user, 2nd Edition (Ljung, L.; 1999) [On the Shelf]," *IEEE Robotics & Automation Magazine*, vol. 19, no. 2, pp. 95-96, 2012.

[68] G. Klančar, A. Zdešar, S. Blažič, and I. Škrjanc, "Motion modeling for mobile robots," *Wheeled Mobile Robotics*, Elsevier, pp. 13-59, 2017.

[69] G. F. Franklin, J.David Powell, and M. L. Workman, *Digital Control of Dynamic Systems*, 3rd ed., 2022. [Online]. Available: https://www.researchgate.net/publication/31849881_Digital_Control_of_Dynamic_Systems-Third_Edition

[70] H. K. Khalil, *Nonlinear Systems*, 3rd ed., Prentice Hall, Upper Saddle River, NJ, 2002.

[71] Hennessy, *Computer Architecture*, 6th ed., Elsevier, 2017.

[72] L. N. Trefethen, *Numerical Linear Algebra: Society for Industrial and Applied Mathematics*, 1997.

[73] Raspberry Pi 4 Model B Rev 1.4 - Geekbench. [Online]. Available: https://browser.geekbench.com/v6/cpu/265980 (accessed: March 7 2025).

[74] Intel Xeon E-2176M Benchmarks - Geekbench. [Online]. Available: https://browser.geekbench.com/processors/intel-xeon-e-2176m (accessed: Mar. 7 2025).

[75] The top 50+ fastest computers in the VMW Research Group. [Online]. Available: https://web.eece.maine.edu/~vweaver/group/machines.html (accessed: Mar. 7 2025).

[76] Intel, Export Compliance Metrics for Intel® Microprocessors. [Online]. Available: https://www.intel.com/content/www/us/en/support/articles/000005755/processors.html (accessed: Mar. 7 2025).

[77] I. Maurovic, M. Baotic, and I. Petrovic, "Explicit model predictive control for trajectory tracking with mobile robots," *Proc. of IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, Budapest, Hungary,, pp. 712-717 2011.

[78] B. Brito, B. Floor, L. Ferranti, and J. Alonso-Mora, "Model predictive contouring control for collision avoidance in unstructured dynamic environments," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4459-4466, 2020.

[79] J. B. Rawlings, M. M. Diehl, and D. Q. Mayne, *Model Predictive Control: Theory, Computation and Design*, 2nd ed., Madison, Nob Hill, 2017.

[80] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," *SIAM Journal on Control and Optimization*, vol. 43, no. 5, pp. 1714-1736, 2005.



Hamza Hobbani received his B.Eng. degree in mechatronics in 2016 and an M.Sc. degree in mechatronics and robotics from the Frankfurt University of Applied Sciences, Frankfurt/Main, Germany in 2018. He is currently working as an industrial robot's software developer at Robert Bosch GmbH conducting his Ph.D. cooperatively between the University of Cadiz, Cadiz, Spain and the Frankfurt University of Applied Sciences. His research focuses on autonomous navigation of mobile robot platforms.



Fernando Perez-Peña received his Engineering degree in telecommunications from the University of Seville, Seville, Spain and his Ph.D. degree (specialized in neuromorphic motor control) from the University of Cadiz, Cadiz, Spain in 2009 and 2014 respectively. In 2015, he was a postdoc at CITEC, Bielefeld University, Bielefeld, Germany. He has been an Associate Professor in the Architecture and Technology of Computers Department of the University of Cadiz since 2019. His research interests include neuromorphic engineering, CPG, motor control, and neurorobotics.



Karsten Schmidt received his Ph.D. degree in nuclear physics in 1995 from Goethe University, Frankfurt/Main, Germany. During his Postdoctoral position he worked at GSI, Darmstadt and the University of Edinburgh. After that he was employed at Continental Automotive Systems, Lufthansa Systems and SAP. From 2005 to 2009, he was a professor for automotive mechatronics at the Coburg University of Applied Sciences. He is currently a professor for mechatronics at Frankfurt University of Applied Sciences. His research interests include control systems, embedded systems, and mobile robotics.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.